

Data Platform Engineer Challenge

The company *Company&Company* needs to start saving billions of social media publications daily from a feed.

Two very different workloads must coexist on the same saved data:

- Client queries: low-latency, high-frequency reads from the *Company&Company* backoffice (search by keyword, filter by author, fetch single publications)
- Internal workloads: the R&D team trains AI models and runs batch analytics over the full dataset; these jobs are long-running and read-heavy.

As a Data Platform Engineer, your job is to design a platform that stores this data where neither workload degrades the other, and where other teams can work with the data without understanding the underlying infrastructure.

Provided

We provide a “producer project”, which is a simple HTTP webhook that emits a continuous stream of publication events. You will find it in the `producer/` folder of this repository. Run it with:

```
python producer.py --url http://localhost:8000/publications
--workers 5
```

(You also have a `consumer.py` that you can use to test the `producer.py`)

Publications follow this schema:

```
class Metrics(BaseModel):
    likes: int
    views: int
    comments: int
    shares: int
    follower_count_at_post: int

class Publication(BaseModel):
    publication_url: HttpUrl
    title: str
    author_name: str
    author_id: UUID
    published_at: datetime
    description: str
    media_url: HttpUrl
    metrics: Metrics
    created_at: datetime
```

```
updated_at: datetime | None = None
deleted_at: datetime | None = None
```

Challenge

1 - Storage Design & Data Ingestion

Design and implement an ingestion pipeline that:

1. Save the publications in a “source-of-truth” database
2. Make the data available to low-latency full-text search on `author\_name`, `description` and `title`, with filters on `author\_id`, `published\_at`, `created\_at`, and metrics

You can use any solution for this (e.g. MongoDB, PostgreSQL, ElasticSearch, Delta Lake, Iceberg, etc.). You need to justify your choices in the README.md.

We must be able to reproduce your solution; You can deliver a Docker Compose, or Kubernetes files (we accept the YAML Kubernetes manifests, or Helm or Flux), or Terraform files for a solution in the cloud.

We provided you with a “producer” project; you can see the schemas there, and you can run the project to create a stream of data to feed the database.

2 - Internal Data API

Build a small FastAPI service that writes and reads the data stored in Part 1:

Endpoint	Description
POST /publications	Insert a publication • If the URL already exists in the database, don't create a new publication; update the existing one
GET /publications/{publication_id}	Fetch a single post by ID from the database
GET /publications/search?q=&...	Full-text search on “description”, with optional filters
GET /authors/{author_id}/stats	Aggregate stats per author: “total posts” and “average engagement rate” • Engagement rate per publication is: $(likes+views+comments+shares) / follower_count_at_post$
• “publication_url” is unique	

- All deleted publications must not appear in search results or stats

Note: You can decide if you want to create only one service to read/write the data, or if you want to create two services, for example, one to read/write the “source of truth” database and another one to query the indexed data. Feel free to take any solution and justify it.

3 - Data quality

Add automated data quality checks that run as part of the ingestion pipeline.

- ``publication_url``, ``author_id``, ``published_at`` are not null - Hard fail
- ``engagement_rate`` is between 0 and 100 - Hard fail
- ``published_at`` is not in the future - Hard fail
- ``published_at`` is older than 24h - Warn only, log and continue
- duplicated ``publication_url`` - Warn only, log and update publication

4 - Schema Evolution

If the product team announces that a new optional field ``platform`` (``instagram | tiktok | youtube | other``) will be added to all future publications. Older publications will not have it.

Also, each platform can introduce new metrics

- Write a “migration plan” in the README.md describing how you would roll out this change without downtime. The API is serving live traffic, and the ingestion pipeline is running continuously
- (Optional but valued) Implement the migration for your local environment and verify backward compatibility

Submission

- 1 - Push your work to a Github or Gitlab repository
- 2 - Include a top-level “README.md” with
 - How to run the full stack locally from scratch
 - Architecture diagram (text-based is fine)
 - Technology choices and trade-offs
 - What would you do differently with more time
- 3 - Give access to the repository for the following emails:
 - rui.martins@primetag.com
 - jose.santos@primetag.com

Good luck! We're excited to learn more about you through this challenge.
If you have any questions, don't hesitate to reach out.